

	  Katedra Grafiki Wizji Komputerowej i Systemów Cyfrowych		RAu6	
Rok akademicki:	Rodzaj studiów*: SSI/NSI/NSM	Przedmiot (Języki Asemblerowe/SMiW):	Grupa	Sekcja
2025/2026	SSI	SMiW	3	ISMIP
Imię:	Szymon	Prowadzący: OA/JP/KT/GD/GB/KH/AO	KT	
Nazwisko:	Mamok			
Report				
Temat projektu: Smart Ball				
Główne założenia projektu: • Urządzenie musi przetrwać ekstremalne przeciążenia występujące podczas uderzenia piłki • System musi bezprzewodowo komunikować się ze smartfonem • Urządzenie musi charakteryzować się niskim poborem prądu, automatycznie przechodząc w stan uśpienia (Light Sleep) po wykryciu bezruchu i wybudzając się przy uderzeniu/poruszeniu. Funkcje urządzenia i aplikacji: • Wykrywanie i analiza uderzeń: Rejestracja przeciążeń rzędu kilkudziesięciu G, obliczanie siły uderzenia, prędkości początkowej, czasu kontaktu ze stopą oraz wskaźnika czystości uderzenia. • Analiza lotu: Obliczanie czasu zawieszenia w powietrzu, prędkości obrotowej w obrotach na minutę (RPM) oraz charakterystyki rotacji • Lokalizacja i telemetria: Przekazywanie współrzędnych GPS na mapę w aplikacji • Archiwizacja sesji: Zapis danych z treningów do plików CSV i generowanie historii zdarzeń.				

Analiza zadania i uzasadnienie wyboru elementów elektronicznych

Konieczne było rozwiązanie problemu różnicy między standardowym ruchem piłki a momentem jej uderzenia. Zwykle czujniki mają zakres pomiarowy do 16G, co jest nie wystarczające przy kopaniu piłki.

Analiza elementów elektronicznych

MCU – ESP32 s3

ESP32 s3	STM32L476RG	nRF52832
• Wysoka wydajność(dwa rdzenie) • Zintegrowane WI-FI i BLE • Częstotliwość 240 MHz • Pobór prądu: 20 mA	• Bardzo oszczędny • Brak komunikacji bezprzewodowej • Częstotliwość 80 MHz • Pamięć flash: 1 MB • Pobór prądu: 10 mA	• Oszczędne • Wydajna komunikacja BLE • Słaba moc obliczeniowa • Częstotliwość 64 MHz • Pamięć flash: 510 KB • Pobór prądu: 8 mA

Akcelerometr wysokich przeciężeń- H3LIS200DL

H3LIS200DL	H3LIS331DL	ADXL372
• Zakres pomiarowy do $\pm 200g$ • ODR 1 kHz • Pobór prądu 300 μA • Dostępny w Polsce	• Zakres pomiarowy do 400g • ODR 1 kHz • Pobór prądu 300 μA • Tani, ale dostępny w Chinach	• Zakres pomiarowy do 200g • Trudno dostępny • ODR 3,2 kHz • Pobór prądu 22 μA

IMU (żyroskop + akcelerometr) - LSM6DSV16X

LSM6DSV16X	MPU6050	BNO055
• Sprzętowy koprocesor SFLP (oblicza kwaterniony w krzemie) • Zakres żyroskopu do ± 4000 dps • Sprzętowe wybudzanie Latched Wake-up • Odświeżanie 960 Hz lub 240 Hz z SFLP • Pobór 0,65mA	• Brak wsparcia dla kwaternionów • Zakres żyroskopu do ± 2000 dps • Odświeżanie 1 kHz lub 200 Hz z DMP • Pobór 3,8 mA	• Niska odporność na wstrząsy 4000g (w porównaniu do 10'000g u konkurencji) • Odświeżanie 100 Hz • Pobór 12 mA

GPS Dual Band (L1 oraz L5) - Allynstar TAU1201

Allynstar TAU1201	Quectel LC29D
• Pobór prądu: 30 mA • Cena: 60 zł	• Pobór prądu 45 mA • Funkcja Dead Reckoning • Cena: 120 zł

Aplikacja mobilna- Kotlin

Natywna aplikacja na Android (Kotlin)	Cross-Platform C# i Xamarin
• Niezawodność stosu BLE • Wydajność UI – możliwość rysowania skomplikowanych wykresów	• Możliwe wąskie gardła w bridge pomiędzy aplikacją a sprzętowym modulem • Duży narzut pamięciowy – ryzyko UI stuttering

Narzędzia użyte w projekcie:

- Kicad



- Fusion 360



- Visual Studio Code



- Esp-idf

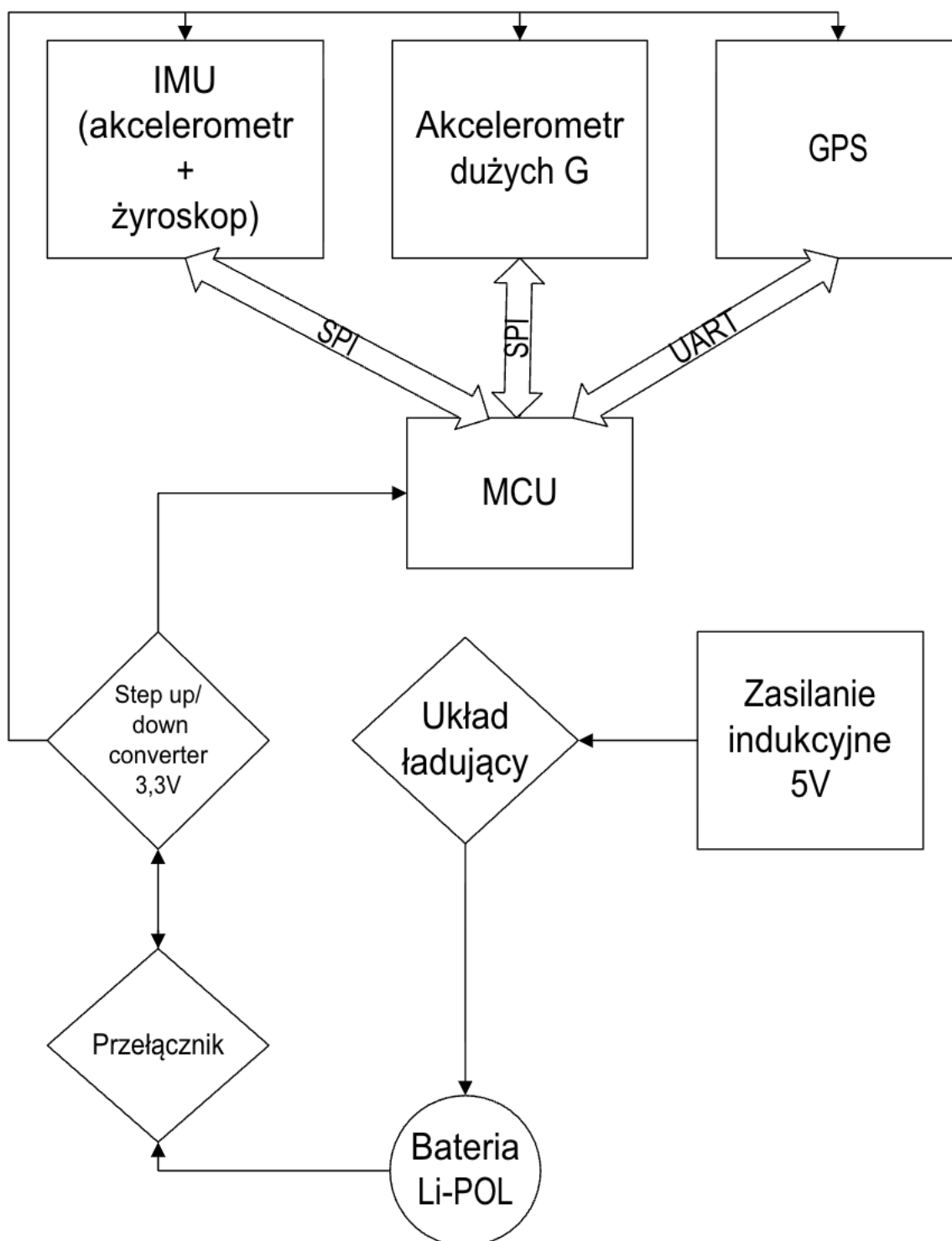


- Android Studio



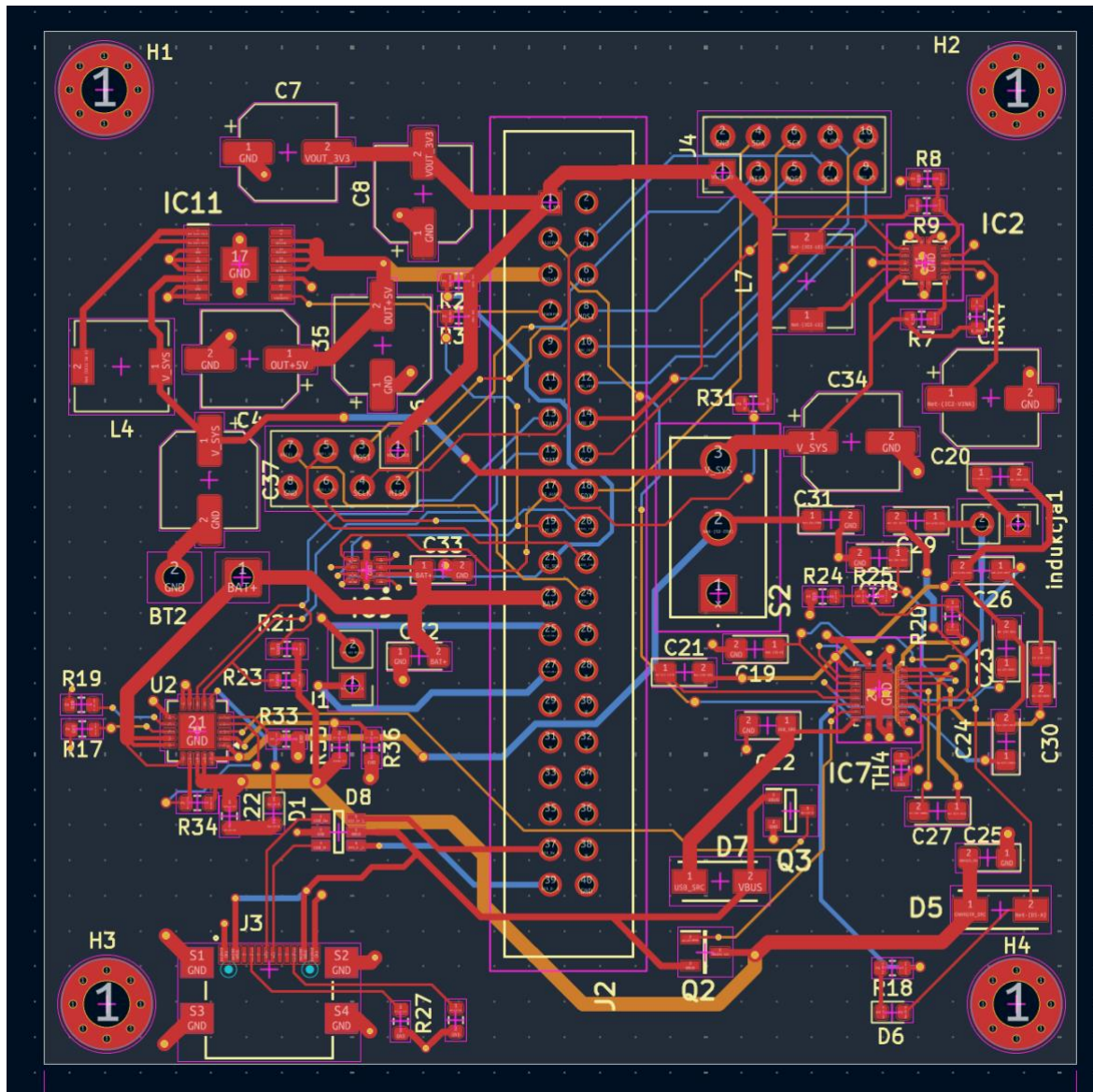
Schemat projektu

Schemat blokowy

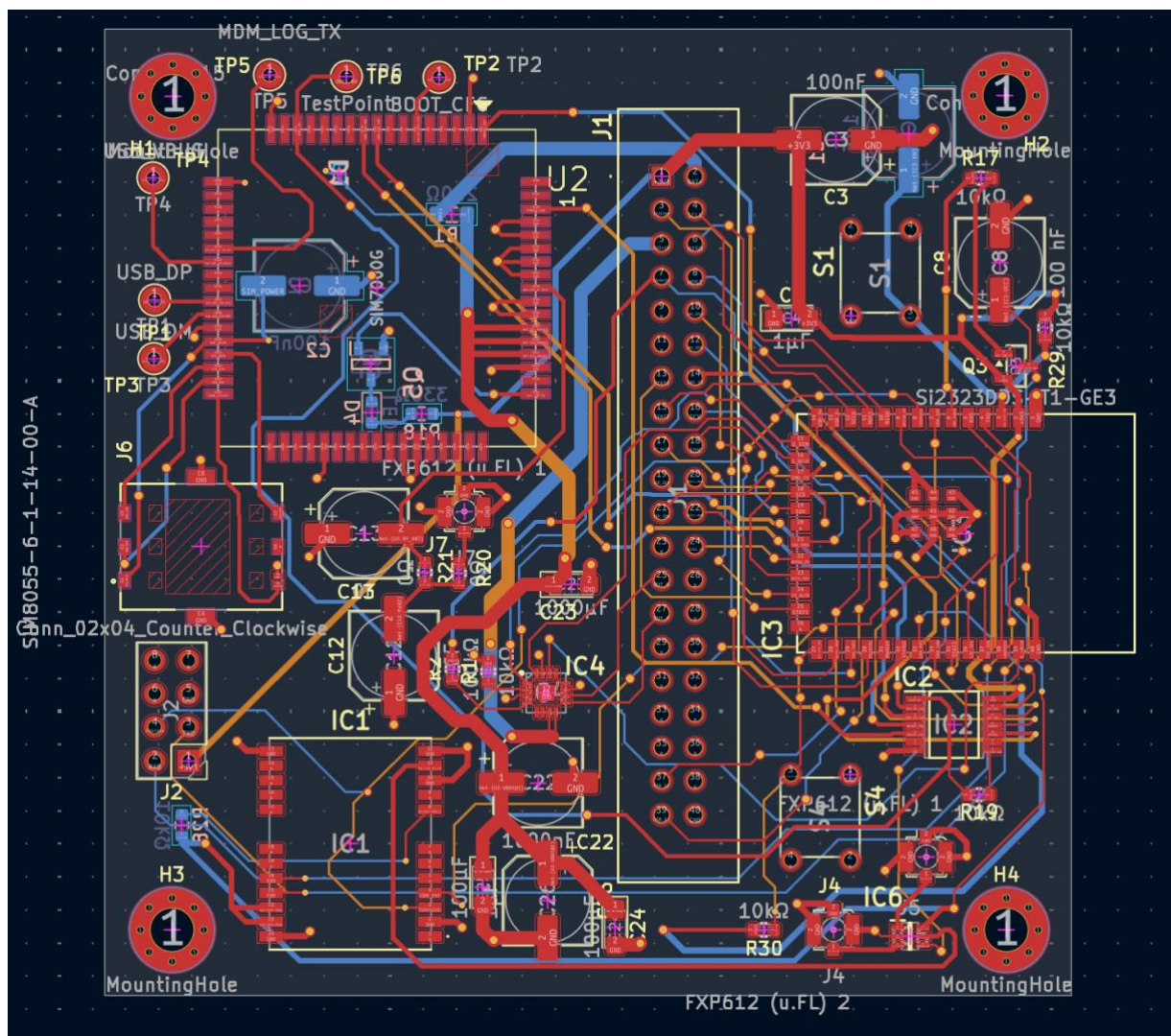


Schemat PCB

Płytki z zasilaniem i czujnikami



Płytki z MCU

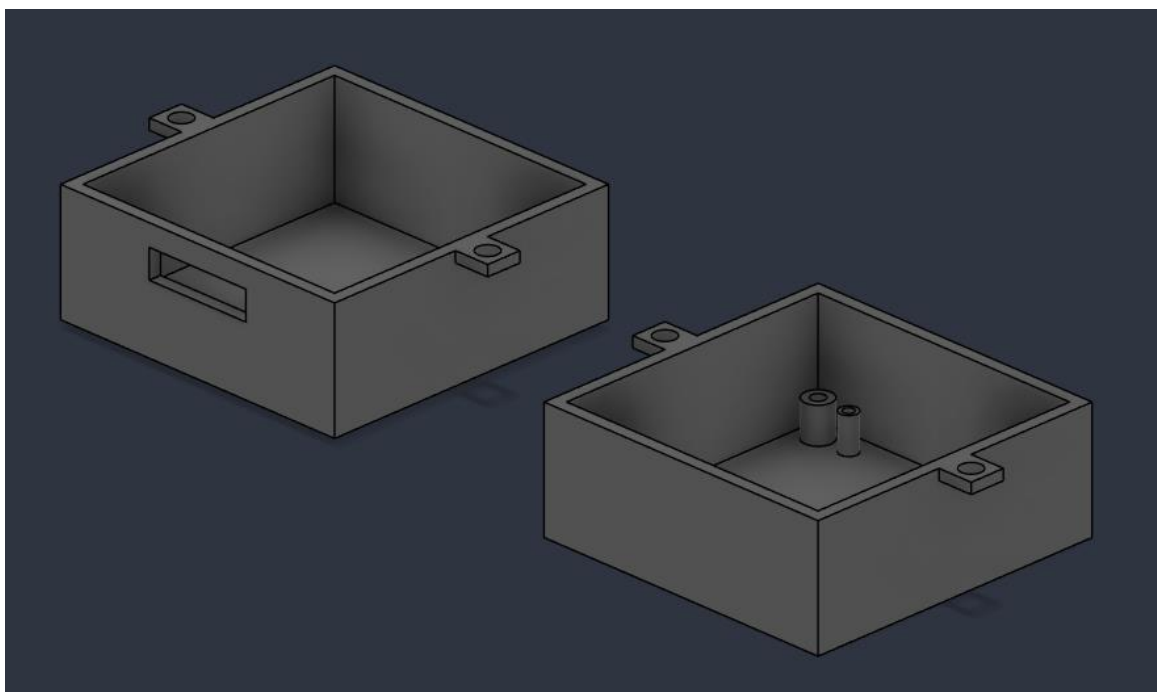


Obudowa:

Rdzeń z elektroniką. Otwór w lewym prostopadłościanie jest przeznaczony na antenę WI-FI/BLE

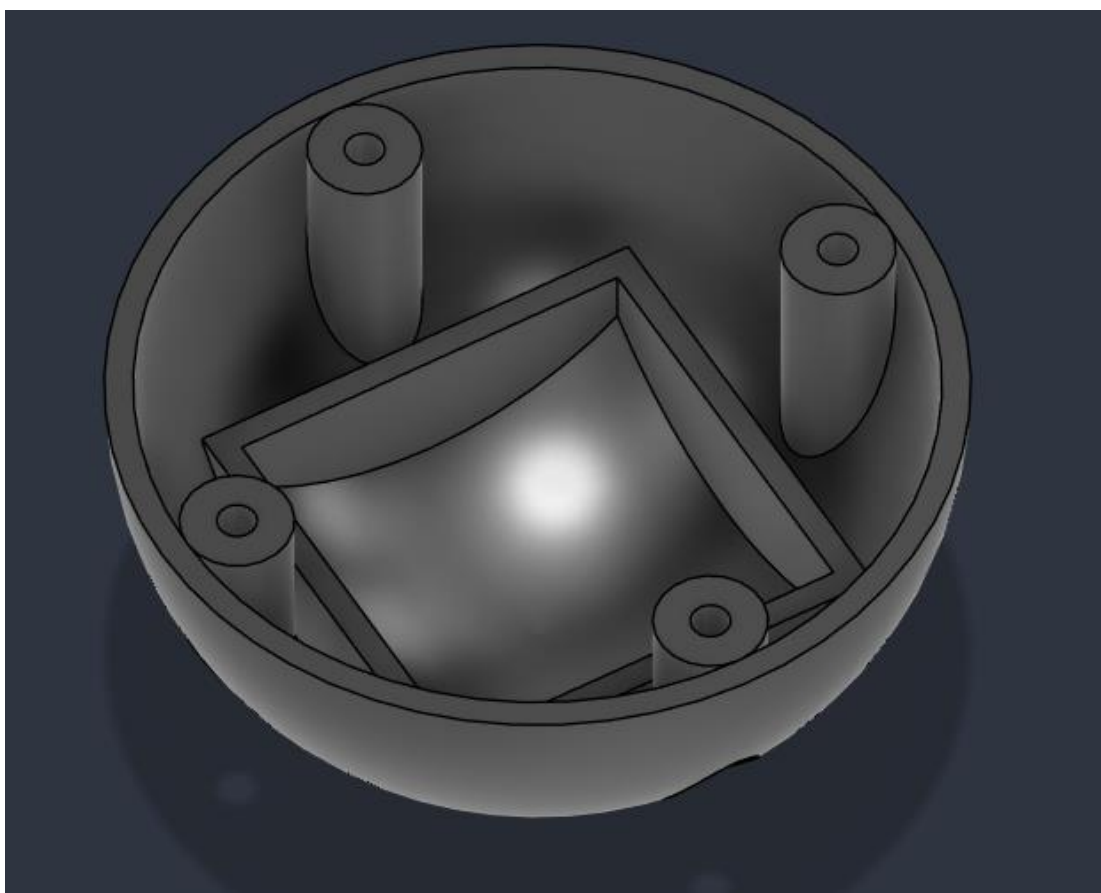
Płytki z zasilaniem jest bezpośrednio przymocowana do rdzenia, aby wiernie odbierać wstrząsy.

Płytki z MCU znajduje się pomiędzy gąbkami amortyzującymi najcięższe przeciążenia.

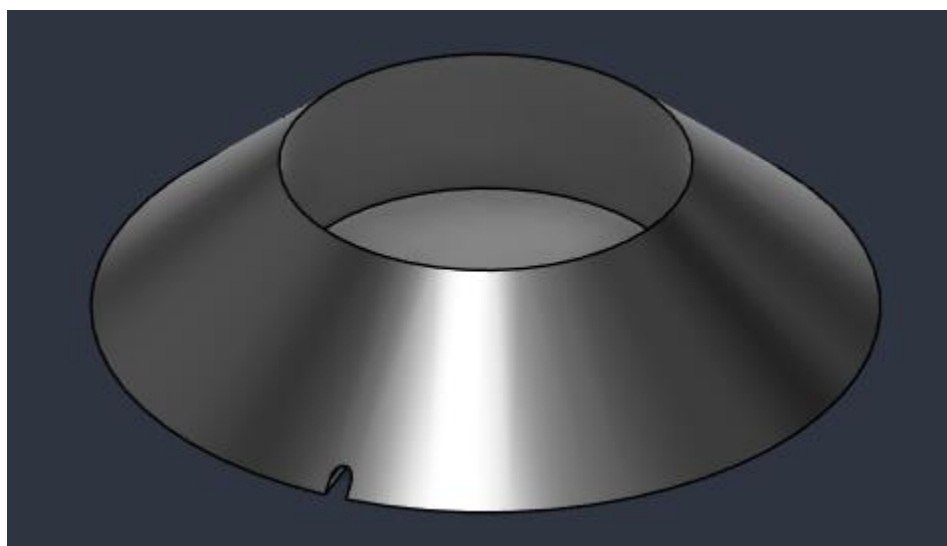


Zewnętrzna część obudowy.

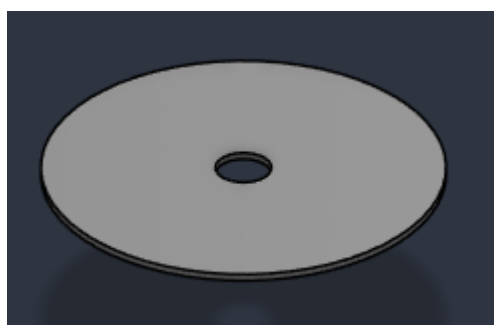
Do jej wnętrza przymocowana jest cewka z zasilania indukcyjnego.



Podstawa pod piłkę wraz z ładowarką indukcyjną.



Ośłona indukcji w podstawie.



Algorytm oprogramowania urządzenia

Smart piłka

Algorytm

1. Inicjalizacja
 - a. Inicjalizacja pamięci trwałej NVS i załadowanie zapisanej konfiguracji (progi wybudzenia, uśpienia, czas bezczynności).
 - b. Utworzenie kolejek FreeRTOS (`data_queue`, `gps_queue`) do asynchronicznej wymiany danych między zadaniami.
 - c. Inicjalizacja stosu Bluetooth (NimBLE) oraz konfiguracja usług i charakterystyk GATT.
 - d. Konfiguracja magistrali SPI oraz UART, a także inicjalizacja czujników
2. Główna pętla
 - a. Odczyt surowych danych z akcelerometru High-G oraz modułu IMU przez SPI.
 - b. Normalizacja wektorów: Obrót lokalnych odczytów do globalnego układu odniesienia przy pomocy kwaternionów pobranych sprzętowo z IMU
 - c. Odczyt ostatniej znanej pozycji GPS zadeklarowanej przez zadanie parsujące UART.
 - d. Analiza bezruchu
 - i. Obliczenie wektora wypadkowego IMU.
 - ii. Gdy czas bezruchu przekroczy ustalony czas układ wchodzi w tryb uśpienia
 - e. Analiza zderzenia
 - i. Jeżeli siła wypadkowa z czujnika High-G przekroczy ustawiony próg
 1. Przekazanie do kolejki `data_queue` serii ostatnich pomiarów przechowywanych w buforze kołowym.
 2. Następnie przekazanie do kolejki pomiaru, który został sklasyfikowany jako uderzenie
 3. Przesyłanie do kolejki serii pomiarów po głównym zdarzeniu.
 - ii. Gdy siła wypadkowa nie przekroczyła progu pomiar zapisywany jest do bufora kołowego, a pojedynczy pomiar jest przekazywany do `data_queue` do 30 ms
3. Równoległe zadanie BLE
 - a. Wysyła dane z kolejki `data_queue`
 - b. Odbiera komunikaty dotyczące zmiany parametrów

Opis kluczowych zmiennych.

- `config_wake_ths_g` (float): Próg przyspieszenia (w G) wybudzający układ ze snu.
- `config_sleep_ths_g` (float): Maksymalna tolerancja odchyłki od 1 G (grawitacji), poniżej której urządzenie jest uznawane za będące w bezruchu.
- `config_idle_time_s` (int): Czas (w sekundach) ciągłego bezruchu wymagany do uśpienia układu.

- `CRASH_THRESHOLD_G` (float): Próg przeciążenia z czujnika High-G oznaczający moment kopnięcia piłki.
- `data_queue`, `gps_queue` (`QueueHandle_t`): Kolejki FreeRTOS przechowujące ramki danych z czujników oraz z modułu GPS.
- `is_phone_connected` (bool): Flaga informująca, czy smartfon jest połączony przez BLE; blokuje uśpienie ESP32 w trakcie aktywnej sesji.
- `pre_hit_buffer` (tablica `global_data_t`): Bufor kołowy przechowujący historię odczytów tuż przed uderzeniem.
- `current_lat`, `current_lon` (float), `current_fix` (bool): Zmienne przechowujące najnowszą, wyparsowaną pozycję i status modułu GPS.
- `global_data_t` (struct): Główna struktura wymiany danych. Agreguje znormalizowane pomiary z akcelerometru High-G, IMU (akcelerometr i żyroskop), stan GPS (szerokość, długość, fix) oraz flagę określającą typ pakietu (lot czy zderzenie). Przesyłana w całości przez BLE do aplikacji.
- `vector3d_t` (struct): Podstawowa struktura matematyczna przechowująca trzy składowe wektora w przestrzeni trójwymiarowej (x, y, z) w postaci liczb zmiennoprzecinkowych (float).
- `accel_data` (struct): Struktura pomocnicza przechowująca surowe, fizyczne odczyty przyspieszenia (osie x, y, z) przekonwertowane na wartość [g], pochodzące z czujnika uderzeniowego H3LIS331DL.
- `imu_data` (struct): Rozbudowana struktura przechowująca kompleksowe odczyty z układu LSM6DSV16X. Zawiera wektory przyspieszenia (accel), rotacji (gyro) oraz wyliczone przez koprocessor SFLP kwaterniony (quat ze składowymi x, y, z, w).
- `sensor_spi_handle_t` (struct): uchwyt dla urządzeń na magistrali SPI. Przechowuje wskaźnik konfiguracyjny interfejsu (`spi_handle`) oraz zdefiniowany numer pinu *Chip Select* (`cs_pin`), pozwalając na szybkie przełączanie między czujnikami.
- `gps_data` (struct): Struktura, pod którą zaalokowano pamięć w kolejce FreeRTOS (`gps_queue`), służąca do wymiany informacji o najnowszej pozycji z asynchronicznie działającym parserem NMEA

Opis funkcji wszystkich procedur

- `app_main()`: Główny punkt wejścia; ładuje konfigurację, tworzy kolejki i uruchamia zadania.
- `ble_data_tx_task()`: Zadanie FreeRTOS pobierające dane z kolejki `data_queue` i wysyłające je jako pakiety powiadomień BLE (Notify) do aplikacji.
- `sensors_set()`: Konfiguruje interfejs SPI oraz ustala parametry (zakresy G i ODR) dla czujników H3LIS200DL i LSM6DSV16X
- `sensors_enter_light_sleep()`: Konfiguruje piny wybudzające, usypia CPU i obsługuje czyszczenie flag przerwań po wybudzeniu.
- `accel_get()`, `imu_get()`: Procedury komunikujące się przez SPI (czytające rejestry czujników) oraz konwertujące surowe dane liczbowe na wartości fizyczne (G, dps, kwaterniony)
- `sensors_reading_task()`: Główna pętla zbierająca dane, wyliczająca czas bezruchu, monitorująca wstrząsy i obsługująca bufor kołowy uderzenia.
- `gps_start()`, `gps_read_task()`: Inicjalizacja magistrali UART i ciągłe parsowanie przychodzących znaków w poszukiwaniu ramek NMEA (\$GPRMC) w celu wyciągnięcia z nich koordynatów.

Opis interakcji oprogramowania z układem elektronicznym

Magistrala SPI: Mikrokontroler wykorzystuje sprzętowy kontroler SPI (`SPI2_HOST`) z mechanizmem DMA do szybkiego, naprzemiennego odpytywania dwóch układów scalonych. Linie *Chip Select* (CS) pozwalają oprogramowaniu na wybór akcelerometru lub żyroskopu. Wysłanie maski 0x80 w pierwszym bajcie ramki mówi czujnikowi, że oprogramowanie żąda odczytu jego wewnętrznych rejestrów.

Przerwania GPIO (Interrupts): Czujniki pracują niezależnie od głównego CPU. Aplikacja konfiguruje układ H3LIS200DL tak, aby po przekroczeniu progu przeciążenia (np. przy poruszeniu) sprzętowo ustawił stan wysoki na swoim wyjściu (INT1). Stan ten wymusza fizyczne wybudzenie rdzenia ESP32 z trybu *Light Sleep* poprzez mechanizm `gpio_wakeup_enable`, minimalizując ingerencję programową podczas uśpienia.

Magistrala UART: Mikrokontroler wykorzystuje kontroler `UART_NUM_1` z przerwaniami do asynchronicznego przechwytywania danych od modułu GPS. System nie musi aktywnie odpytywać nadajnika – znaki NMEA są "wpychane" do bufora sprzętowego UART, a zadanie RTOS (`uart_read_bytes`) parsuje je dopiero, gdy są dostępne.

Stos Radiowy (NimBLE): Komunikacja bezprzewodowa integruje warstwę programową aplikacji ze sprzętowym radiem 2.4 GHz mikrokontrolera.

Aplikacja mobilna

Algorytm

Algorytm pracy aplikacji na systemie Android opiera się na architekturze sterowanej zdarzeniami (event-driven)

1. Rutynowy odbiór danych

- a. Gdy zostanie odebrana paczka od ESP32, uruchamiany jest asynchroniczny callback `onCharacteristicChanged`.
- b. Surowy ciąg bajtów jest parsowany. Aplikacja rozpoznaje, czy jest to ramka konfiguracyjna (tekstowa), czy ramka pomiarowa z czujników.
- c. Zdekodowane dane wyzwalają funkcje zwrotne (tzw. listenery), które przekazują te wartości do `MainActivity` w celu płynnego narysowania wykresów
- d. Równolegle, odebrana próbka wysyłana jest do pliku CSV (jeśli włączony jest trening)

2. Interakcja z użytkownikiem

- a. Historia i Detale: Z poziomu historii zdarzeń użytkownik może wybrać konkretne uderzenie. Aplikacja przekazuje wybrany wycinek danych do `ShotDetailsActivity`, gdzie następuje głęboka analiza fizyczna zderzenia (obliczanie prędkości, rotacji, czasu kontaktu).
- b. Lokalizacja Hybrydowa: W widoku `LocationActivity` aplikacja nanosi pozycję piłki na mapę OSMDroid, korzystając ze współrzędnych odbieranych przez BLE. W przypadku utraty zasięgu Bluetooth, aplikacja automatycznie łączy się z brokerem MQTT i odbiera lokalizację GSM przesyłaną z chmury, pozwalając na zdalne włączenie w piłce buzera ułatwiającego poszukiwania.

Opis kluczowych zmiennych

- `handleIncomingData(bytes: ByteArray)` (`BleManager`): Dekoduje surową tablicę bajtów przychodzącą z BLE za pomocą obiektu `ByteBuffer` ustawionego na architekturę `LITTLE_ENDIAN`. Rozbija paczkę na pojedyncze liczby

zmiennoprzecinkowe (składowe przyspieszeń High-G, IMU, żyroskopu oraz koordynaty GPS) i propaguje je do interfejsu graficznego oraz menedżera sesji.

- `calculateExitVelocityKmH(hitSamples: List<SampleData>)` (MetricsCalculator): Funkcja analityczna, która metodą numerycznego całkowania pola pod wykresem przyspieszenia ($dV = a * dt$) wylicza prędkość początkową piłki. Aby uniknąć zawyżania prędkości przez szumy z otoczenia, ignoruje próbki, w których przyspieszenie wypadkowe jest mniejsze niż $\sim 1.5G$ (15 m/s^2).
- `determineSpinAxisAndMagnus(samples: List<SampleData>)` (MetricsCalculator): Określa fizyczną charakterystykę rzutu (Top-spin, Back-spin, Side-spin, Rifle-spin) porównując uśrednione moduły prędkości obrotowej z żyroskopu na osiach X, Y i Z z okresu swobodnego lotu piłki.
- `syncCharts(src: LineChart, dst: LineChart)` (MainActivity): Procedura sprzęgająca gesty dotykowe. Przechwytuje akcję przybliżania lub przesuwania jednego wykresu, wyciąga jego macierz transformacji (Matrix) i aplikuje te same wartości przesunięć do drugiego wykresu. Dzięki temu wykres akcelerometru i żyroskopu zawsze pokazują ten sam fragment czasu.
- `logSampleToCurrentSession(...)` (SessionManager): Moduł zapisu logów. Konsoliduje otrzymane dane do formatu tekstowego CSV (gdzie kolumny oddzielone są przecinkami), dodaje unikalny stempel czasowy i poprzez dopisywanie na końcu pliku utrzuca ciągły strumień telemetrii treningu.
- `accelChart, gyroChart` (LineChart): Obiekty biblioteki MPAndroidChart renderujące na żywo wieloliniowe wykresy przyspieszeń i rotacji.
- `gattCallback` (BluetoothGattCallback): Obiekt nasłuchujący zdarzeń BLE, obsługujący połączenie, zmianę MTU i asynchroniczny strumień danych z piłki.
- `HistoryEntry` (Data Class): Struktura grupująca całą listę pojedynczych próbek w jedno zdarzenie (uderzenie lub lot) wraz z wyliczoną wartością szczytową.
- `SampleData` (Data Class): Struktura reprezentująca pojedynczą "klatkę" pomiarową w czasie (3 osie akcelerometru, 3 osie żyroskopu i znacznik czasu).
- `buffer` (ByteBuffer): Bufor skonfigurowany w trybie LITTLE_ENDIAN, dekodujący surowy ciąg bajtów z BLE na liczby zmiennoprzecinkowe.
- `onDataSampleReceivedListener` (Lambda): Callback (funkcja zwrotna) przekazująca zdekodowane pomiary z menedżera BLE do wątku głównego (UI) w celu rysowania wykresów.

- fullHitHistory (mutableListOf): Globalna lista przechowująca historię wszystkich zarejestrowanych zdarzeń (uderzeń i swobodnych lotów) z bieżącego treningu.

Opis funkcji wszystkich procedur.

- calculateExitVelocityKmH(): Wylicza prędkość początkową piłki w km/h, wykorzystując numeryczne całkowanie pola pod wykresem przyspieszenia (odrzuca szumy poniżej $\sim 1.5G$).
- determineSpinAxisAndMagnus(): Kategoryzuje fizyczną oś rotacji piłki podczas lotu (Top-spin, Back-spin, Side-spin, Rifle-spin) na podstawie średnich wartości z żyroskopu.
- calculateHangTimeSeconds(): Oblicza czas lotu zliczając próbki, w których wypadkowe przyspieszenie IMU odpina grawitację (spada poniżej 300 mg).
- calculateContactTimeMs() i calculateSmashFactor(): Określają czas kontaktu stopy z piłką oraz czystość uderzenia wyliczoną ze stosunku szczytowej siły G do czasu kontaktu.
- calculateImpactAngle(): Wykorzystuje funkcję arkus tangens na wektorach uderzeniowych w celu obliczenia azymutu i kąta podbicia strzału.
- handleIncomingData(): Główne serce przetwarzania radiowego. Dekoduje surową tablicę bajtów z ESP32 za pomocą bufora (zgodnego z architekturą LITTLE_ENDIAN) i rozbija ją na pojedyncze liczby zmiennoprzecinkowe dla poszczególnych osi.
- startScanAndConnect(): Uruchamia sprzętowy skaner BLE, filtruje w poszukiwaniu urządzenia "SmartBall-Config" i inicjuje połączenie.
- sendCommand(): Asynchronicznie przesyła do mikrokontrolera komendy tekstowe (np. ustalające progi wybudzenia) za pośrednictwem charakterystyki GATT.
- syncCharts(): Zabezpieczona flagą metoda, która odczytuje macierz transformacji z jednego wykresu podczas gestu użytkownika (np. przybliżenia) i aplikuje ją na drugi wykres, zapewniając ich idealną synchronizację w czasie.
- addSampleToChart(): Odbiera zdekodowane odczyty w wątku w tle, wylicza siły wypadkowe i odświeża interfejs rysujący na żywo za pomocą funkcji runOnUiThread.
- updateMapPosition(): Inicjalizuje i aktualizuje pozycję znacznika (Markera) na geograficznej mapie OSMDroid przy każdej zdekodowanej paczce z nowymi współrzędnymi GPS.

- `drawCollisionChart()`: Wyodrębnia szczegółowy zbiór próbek dla konkretnego, historycznego strzału i renderuje jego krzywą siły uderzenia i rotacji na jednym wykresie z podwójną osią Y.
- `startNewSession()` i `stopCurrentSession()`: Tworzy na smartfonie dedykowany folder z unikalnym stemplem czasowym oraz otwiera (lub zamyka) strumień zapisu do pliku `telemetry_data.csv`.
- `logSampleToCurrentSession()`: Formatuje spływające odczyty (`h3x`, `ax`, `lat`, `lon` itd.) do formatu tekstowego rozdzielanego przecinkami (CSV) i dopisuje na bieżąco nowy wiersz do pliku.

Specyfikacja zewnętrzna

Z uwagi na docelowe przeznaczenie (wnętrze piłki sportowej), fizyczne urządzenie pozbawione jest jakichkolwiek klasycznych, mechanicznych przycisków sterujących. Całkowite sterowanie odbywa się zdalnie za pośrednictwem aplikacji mobilnej, która oferuje interfejs do zmiany parametrów w locie:

- Próg wybudzenia
- Próg uśpienia
- Czas bezczynności
- Próg zderzenia
- Zarządzanie sesją

Reakcji oprogramowania na zdarzenia zewnętrzne

Kopnięcie piłki (Przekroczenie `CRASH_THRESHOLD_G`) - resetuje liczniki czasu bezruchu, zrzuca do kolejki nadawczej zbuforowaną historię odczytów, a następnie opóźnia pętlę i przesyła serię ramek pokolizyjnych.

Wejście w stan bezruchu: Po upływie określonego czasu bezruchu oprogramowanie automatycznie przygotowuje czujnik do sprzętowego wybudzania, blokuje pętlę odczytów i wywołuje polecenie wprowadzając mikrokontroler w sen.

Poruszenie uśpionej piłki: Sprzętowy układ czujnika akcelerometru wykrywa anomalię przestrzenną i generuje stan wysoki na swoim pinie. Ten impuls trafia na zadeklarowany pin

GPIO układu ESP32 natychmiastowo przywracając oprogramowanie do pełnego działania i kasując przerwanie w rejestrze akcelerometru.

Utrata połączenia Bluetooth: Rozłączenie ze smartfonem wyzwała zdarzenie BLE_GAP_EVENT_DISCONNECT na układzie ESP32, po czym piłka automatycznie wznawia proces rozgłaszania, oczekując na ponowne parowanie. W tym samym czasie w aplikacji wyzwalany jest callback, który zmienia tekst statusowy.

Opis montażu i uruchamiania

1. Rozkręć obudowę za pomocą imbusa I nasadki
2. Rozkręć rdzeń piłki - (sześcienna kostka)
3. Podnieś jedną część obudowy rdzenia
4. Zlokalizuj przełącznik na płytce I przełącz ją na pozycję ON.
5. Zamknij I przykręć rdzeń
6. Podłącz kabel zasilania doprowadzony do rdzenia urządzenia z kopuły.
7. Włóż rdzeń do kopuły i przykręć obie kopuły ze sobą.
8. Zainstaluj aplikację mobilną
9. Włącz Bluetooth I zezwól aplikacji na dostęp do niego
10. Jeżeli aplikacja nie umie połączyć się z piłką potrząśnij piłką.

Instrukcja obsługi urządzenia

1. Włącz i zezwól: Uruchom aplikację na smartfonie. Upewnij się, że masz włączony Bluetooth oraz moduł Lokalizacji (aplikacja automatycznie poprosi o brakujące uprawnienia do skanowania).
2. Połącz się z piłką: Potrząśnij piłką a następnie przejdź do aplikacji. Aplikacja w sposób niewidoczny dla użytkownika przeskanuje otoczenie. Gdy znajdzie urządzenie "SmartBall-Config", automatycznie nawiąże połączenie – potwierdzeniem będzie zielony pasek statusu w górnej części ekranu.
3. Konfiguracja wstępna: Przejdź do panelu "Ustawienia". Możesz tam odczytać bieżące parametry wpisane w pamięć urządzenia (m.in. progi wybudzenia i snu) oraz w razie potrzeby zaktualizować je nowymi wartościami.

4. Rozpocznij trening: Przejdź do okna "Sesje / Trening". Nadaj sesji nazwę i kliknij "ROZPOCZNIJ NOWY TRENING". Pomiar będzie od tej chwili rejestrowany do ukrytego pliku .csv.
5. Graj: Ekran główny na bieżąco prezentuje wykresy przyspieszeń i rotacji w postaci płynnie przesuwających się linii. Wykonuj swobodne rzuty i uderzenia.
6. Analizuj strzały: Po wykonaniu uderzenia wejdź w zakładkę "Historia". Będą tam wylistowane poszczególne rzuty i kopnięcia (czerwone tło oznacza zderzenie, niebieskie swobodny lot). Kliknięcie w zdarzenie otwiera ekran detali ze statystykami prędkości wyjścia (w km/h), wskaźnikiem smash factor czy wyliczonym z żyroskopu efektem magnusa (rodzaj rotacji).
7. Zlokalizuj zgubę: Jeżeli nie wiesz, gdzie jest piłka, otwórz moduł "Lokalizacja". Ekran wyświetli jej pozycję na mapie bazując na wbudowanym odbiorniku GPS.
8. Zakończ i wyeksportuj: W zakładce z sesjami, po skończonej grze zakończ aktualny trening. Zebraną telemetrię możesz udostępnić jako załącznik pliku .csv, używając przycisku "UDOSTĘPNIJ CSV"

Problemy jakie wystąpiły podczas montażu i uruchamiania

Problem ze zbyt długą listwą IDC: Standardowa taśma ze złączem IDC okazała się zbyt długa, aby zmieścić ją w zwartej obudowie wewnątrz piłki. Próba wykonania i własnoręcznego oprawienia skróconej taśmy IDC doprowadziła początkowo do braku pełnego styku.

- Rozwiązanie: Problem rozwiązano poprzez bardzo staranne, mechaniczne docięnięcie pinów dedykowaną zaciskarką oraz dokładne sprawdzenie ciągłości obwodów miernikiem, co wyeliminowało luzy zagrażające przerwaniem komunikacji podczas wstrząsów.

Resetowanie się układu ESP32 przy uruchamianiu radia Bluetooth: Podczas testów oprogramowania mikrokontroler resetował się samoczynnie (tzw. *Brown-out Reset*) w momencie wywołania inicjalizacji modułu BLE i rozgłaszania radiowego. Było to spowodowane nagłym poborem prądu przez antenę i wewnętrzny wzmacniacz RF, co powodowało chwilowe spadki napięcia.

- Rozwiązanie: Zastąpiono użyte kondensatory w przetwornicy podwyższająco-obniżającej (buck-boost converter) precyzyjnymi odpowiednikami ściśle zalecanymi

przez producenta przetwornicy w nocie katalogowej, a ponadto dołożono dodatkowy kondensator filtrujący o pojemności 47 μF tuż przy zasilaniu ESP32, co zbuforowało piki prądowe.

Zbyt wysokie napięcie na stabilizatorze (Buck up/down converter): Pomiary wyjścia układu zasilania, który miał zapewniać poziom 3.3V, wskazały zbyt wysokie napięcie, co groziło natychmiastowym spalaniem cyfrowych czujników i mikrokontrolera.

- Rozwiązanie: Przeprowadzono inspekcję dzielnika napięcia przy sprzężeniu zwrotnym przetwornicy. Okazało się, że wartość rezystora na linii VINA była nieodpowiednia. Wymiana tego rezystora z wartości 100 k Ω na prawidłową wartość 100 Ω natychmiast przywróciła stabilne napięcie wyjściowe rzędu 3.3V.

Testy poprawności działania urządzenia

Testy łączności i przepustowości BLE: Połączono urządzenie ze smartfonem i zweryfikowano stabilność przesyłania danych pomiarowych z założonym interwałem 30 milisekund (około 33 próbki na sekundę). Weryfikowano brak zgubionych ramek w ciągłym strumieniu danych.

Testy fuzji czujników i odczytów statycznych: Położono uruchomiony układ płasko na stole, a aplikacja w telefonie potwierdziła, że dzięki sprzętowym kwaternionom z IMU i algorytmom oprogramowania, przeliczony wektor grawitacyjny w osi Z prawidłowo wynosił 1G, a osie X i Y wskazywały 0G.

Testy detekcji uderzeń i zrzutowe (Drop-Testy): Przeprowadzono fizyczne testy zrzucania i uderzania urządzenia w celu weryfikacji, czy czujnik High-G poprawnie rejestruje przekroczenie określonej siły (zdefiniowanej zmienną `CRASH_THRESHOLD_G`). Sprawdzone mechanizm bufora kołowego (`pre_hit_buffer`), potwierdzając, że uderzenie inicjuje wysyłkę próbek zarejestrowanych tuż przed pikiem zderzenia.

Testy zarządzania energią: Upewniono się, że po upływie czasu zdefiniowanego jako `config_idle_time_s` (brak ruchu i rotacji), urządzenie automatycznie przechodziło w stan *Light Sleep*. Następnie potrząsano płytką w celu wyzwolenia sprzętowego przerwania wybudzającego (z pinu INT1).

Testy modułu GPS: Przetestowano urządzenie na zewnątrz pod kątem dekodowania ramek NMEA (\$GPRMC) przez interfejs UART i przesyłania dokładnych danych geograficznych do oprogramowania Android (w celu naniesienia znacznika na mapę).

Wnioski

Zasilanie jest kluczem w systemach radiowych: Problemy napotkane podczas uruchamiania jednoznacznie pokazują, że układy mikrokontrolerów z wbudowanym radiem (jak ESP32) są ekstremalnie wrażliwe na projekt sekcji zasilania. Nawet drobna zmiana parametrów (brak kondensatora 47 μ F lub błędny rezystor w sekcji zasilania logiki) natychmiast destabilizuje system lub uniemożliwia wynegocjowanie połączenia BLE. Staranny dobór komponentów a także ich jak najbliższe umiejscowienie są kluczowe do prawidłowej pracy układów.